

IF2211 - Strategi Algoritma

Laporan Tugas Besar 3

Pemanfaatan Pattern Matching untuk Membangun Sistem ATS

(Applicant Tracking System) Berbasis CV Digital



Disusun Oleh:

Jessica Allen	13523059
Anella Utari Gunadi	13523078
Naomi Risaka Sitorus	13523122

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2025

DAFTAR ISI

DAFTAR ISI.....	2
BAB 1.....	4
DESKRIPSI TUGAS.....	4
BAB 2.....	6
LANDASAN TEORI.....	6
2.1 Algoritma Knuth-Morris-Pratt.....	7
2.2. Algoritma Boyer-Moore.....	8
2.3. Algoritma Aho-Corasick.....	10
2.4 Algoritma Levenshtein Distance.....	11
2.5. Aplikasi yang Dibangun (Matchify).....	13
BAB 3.....	15
ANALISIS PEMECAHAN MASALAH.....	15
3.1. Langkah-Langkah Pemecahan Masalah.....	15
3.2. Pemetaan Masalah Menjadi Elemen-Elemen Algoritma KMP.....	15
3.3. Pemetaan Masalah Menjadi Elemen-Elemen Algoritma BM.....	16
3.4. Pemetaan Masalah Menjadi Elemen-Elemen Algoritma Levenshtein.....	16
3.5. Pemetaan Masalah Menjadi Elemen-Elemen Algoritma Aho-Corasick.....	17
3.6. Fitur Fungsional.....	17
3.7. Arsitektur Aplikasi.....	18
3.8. Contoh Ilustrasi Kasus.....	20
BAB 4.....	21
IMPLEMENTASI DAN PENGUJIAN.....	21
4.1. Spesifikasi Teknis Program Database.....	21
4.1.1. Struktur Data.....	21
4.1.2. Fungsi dan Prosedur.....	21
4.2. Spesifikasi Teknis Program Backend.....	22
4.2.1. Struktur Data.....	22
4.2.2. Fungsi dan Prosedur.....	23
4.3. Spesifikasi Teknis Program Frontend.....	25
4.3.1. Struktur Data.....	25
4.3.2. Fungsi dan Prosedur.....	25
4.4. Tata Cara Penggunaan Program.....	26
4.4.1. Fitur-Fitur Frontend.....	26
4.4.2. Search Interface.....	31
4.4.3. CV Summary View.....	33
4.5. Pengujian.....	37
4.5.1. Pengujian 1.....	37

4.5.2. Pengujian 2.....	38
4.5.3. Pengujian 3.....	39
4.5.4. Pengujian 4.....	41
4.6. Analisis Hasil Pengujian.....	42
BAB 5.....	44
KESIMPULAN, SARAN, DAN REFLEKSI.....	44
5.1. Kesimpulan.....	44
5.2. Saran.....	44
5.3. Refleksi.....	44
LAMPIRAN.....	45
Tautan Repository GitHub.....	46
Tautan Video.....	46
DAFTAR PUSTAKA.....	47

BAB 1

DESKRIPSI TUGAS



Gambar 1. CV ATS dalam Dunia Kerja

(Sumber: <https://www.antaranews.com/>)

Di era digital ini, keamanan data dan akses menjadi semakin penting. Perkembangan proses rekrutmen tenaga kerja telah mengalami perubahan signifikan dengan memanfaatkan teknologi untuk meningkatkan efisiensi dan akurasi. Salah satu inovasi yang menjadi solusi utama adalah Applicant Tracking System (ATS), yang dirancang untuk mempermudah perusahaan dalam menyaring dan mencocokkan informasi kandidat dari berkas lamaran, khususnya Curriculum Vitae (CV). ATS memungkinkan perusahaan untuk mengelola ribuan dokumen lamaran secara otomatis dan memastikan kandidat yang relevan dapat ditemukan dengan cepat.

Meskipun demikian, salah satu tantangan besar dalam pengembangan sistem ATS adalah kemampuan untuk memproses dokumen CV dalam format PDF yang tidak selalu terstruktur. Dokumen seperti ini memerlukan metode canggih untuk mengekstrak informasi penting seperti identitas, pengalaman kerja, keahlian, dan riwayat pendidikan secara efisien. Pattern matching menjadi solusi ideal dalam menghadapi tantangan ini.

Pattern matching adalah teknik untuk menemukan dan mencocokkan pola tertentu dalam teks. Dalam konteks ini, algoritma Boyer-Moore dan Knuth-Morris-Pratt (KMP) sering

digunakan karena keduanya menawarkan efisiensi tinggi untuk pencarian teks di dokumen besar. Algoritma ini memungkinkan sistem ATS untuk mengidentifikasi informasi penting dari CV pelamar dengan kecepatan dan akurasi yang optimal.

Di dalam Tugas Besar 3 ini, Anda diminta untuk mengimplementasikan sistem yang dapat melakukan deteksi informasi pelamar berbasis dokumen CV digital. Metode yang akan digunakan untuk melakukan deteksi pola dalam CV adalah Boyer-Moore dan Knuth-Morris-Pratt. Selain itu, sistem ini akan dihubungkan dengan identitas kandidat melalui basis data sehingga harapannya terbentuk sebuah sistem yang dapat mengenali profil pelamar secara lengkap hanya dengan menggunakan CV digital.

BAB 2

LANDASAN TEORI

Pattern matching adalah teknik fundamental dalam *computer science* untuk menemukan dan mencocokkan pola tertentu dalam teks atau data. Dalam konteks *string matching*, *pattern matching* bertujuan untuk mencari kemunculan suatu *pattern* (pola) P dengan panjang m karakter di dalam teks T dengan panjang n karakter, biasanya $m \ll n$.

Algoritma *pattern matching* yang efisien harus mempertimbangkan *trade-off* antara *preprocessing time*, *searching time*, dan *space complexity*. Algoritma *brute-force* memiliki kompleksitas $O(nm)$ dalam kasus terburuk, sementara algoritma yang lebih canggih dapat mencapai $O(n+m)$ atau bahkan sublinear dalam kasus rata-rata.

Terdapat beberapa klasifikasi *string matching*, yaitu *exact string matching*, *approximate string matching*, dan *regular expression matching*. *Exact string matching* mencari kemunculan *pattern* yang identik dengan teks. Terdapat dua jenis *exact string matching*, yaitu *single pattern matching* dan *multiple pattern matching*. Pada *single pattern matching*, satu *pattern* dicari dalam teks, sedangkan pada *multiple pattern matching*, beberapa *pattern* dicari secara simultan.

Klasifikasi kedua, *approximate string matching*, mencari *pattern* dengan toleransi error. *Pattern matching* ini dibagi menjadi tiga jenis, yaitu *edit distance based*, *hamming distance based*, dan *longest common subsequence based*. Pada *edit distance based*, digunakan operasi *insert*, *delete*, dan *substitute*. Pada *hamming distance based*, hanya operasi *substitute* yang digunakan untuk string yang sama panjang. Pada *longest common subsequence based*, dicari kesamaan subsequence terpanjang pada string.

Terakhir, yaitu klasifikasi *regular expression matching*, yang mencari *pattern* berdasarkan *grammar rules*. Klasifikasi ini terbagi menjadi dua jenis, yaitu *finite automata based*, yang menggunakan *state machine*, kemudian juga ada *backtracking based*, yang menggunakan *recursive pattern exploration*.

Pattern matching dapat diaplikasikan dalam berbagai bidang di kehidupan, seperti *information retrieval*, *bioinformatics*, *network security*, *data processing*, *natural language processing*, *software engineering*, *digital forensics*, dan lainnya. Terdapat lima karakteristik algoritma *pattern matching*, yaitu *preprocessing requirement* (apakah diperlukan *preprocessing* *pattern* atau teks), *online vs offline* (dapat bekerja dengan *streaming* data atau diperlukan seluruh

teks), *space complexity* (kebutuhan memori tambahan untuk *preprocessing*), *adaptability* (kemampuan untuk mencari *pattern* termodifikasi tanpa *preprocessing* ulang), dan *worst-case vs average-case performance* (performanya dalam kondisi yang berbeda).

Dalam konteks *Applicant Tracking System* (ATS) seperti pada tugas besar ini, *pattern matching* digunakan untuk mencari kata kunci yang relevan dalam dokumen CV. Hal ini tentunya mempermudah pembaca CV karena memungkinkan *screening* otomatis kandidat berdasarkan kualifikasi teknis yang dibutuhkan.

2.1 Algoritma Knuth-Morris-Pratt

Algoritma Knuth-Morris-Pratt (KMP) adalah algoritma pencocokan string yang dikembangkan oleh Donald Knuth, James H. Morris, dan Vaughan Pratt pada tahun 1977. Algoritma ini merupakan perbaikan dari algoritma *brute-force* dengan mengeliminasi pemeriksaan karakter yang tidak perlu melalui *preprocessing pattern*. Algoritma *brute-force string matching* memiliki kompleksitas waktu $O(nm)$ pada worst case karena setiap terjadi ketidakcocokan, pencarian dimulai dari posisi berikutnya di teks tanpa memanfaatkan informasi yang telah diperoleh. KMP mengatasi masalah ini dengan memanfaatkan informasi *partial match* untuk menghindari *backtracking* pada teks.

Terdapat beberapa konsep fundamental dalam algoritma KMP, yaitu *longest proper prefix* yang juga *suffix* (LPS), *border* (pinggiran), dan *failure function*. LPS terdiri dari *proper prefix* dan *proper suffix*. *Proper prefix* adalah *prefix* yang bukan string itu sendiri, sedangkan *proper suffix* adalah *suffix* yang bukan string itu sendiri. LPS adalah panjang *longest proper prefix* dari *pattern* $[0..i]$ yang juga merupakan *suffix* dari *pattern* $[0..i]$.

Border dari string S adalah substring yang merupakan *prefix* dan *suffix* dari S . *Longest border* digunakan untuk menentukan posisi lanjutan saat ketidakcocokan, sedangkan *failure function* merupakan fungsi yang menyimpan informasi LPS untuk setiap posisi dalam *pattern* dan digunakan untuk menentukan posisi *restart* pencarian ketika terjadi masalah.

Terdapat dua fase dalam algoritma pencarian KMP, yaitu fase *preprocessing* dan fase *searching*. Pada fase *preprocessing*, dihitung *failure function* untuk *pattern* dengan kompleksitas $O(m)$. Fase ini tidak bergantung pada teks yang akan dicari dan hasil *preprocessing* dapat digunakan untuk *multiple text*. Fase *searching* pada KMP dilakukan dari kiri ke kanan tanpa

backtracking pada teks. Ketika terjadi ketidakcocokan di posisi j , digunakan *failure function* untuk menentukan posisi *restart* pencarian. Kompleksitas pencarian ini adalah $O(n)$.

Pada algoritma KMP, teks tidak pernah dibaca mundur dan setiap karakter teks diperiksa maksimal sekali. Selain itu, pattern dapat bergeser lebih dari satu posisi saat terjadi ketidakcocokan. Adapun kompleksitas dari algoritma ini adalah sebagai berikut.

- Kompleksitas Waktu : $O(n+m)$ untuk preprocessing dan searching
- Kompleksitas Ruang : $O(m)$ untuk menyimpan failure function
- Kasus Terburuk : $O(n+m)$
- Kasus Terbaik : $O(n+m)$

Keunggulan dari algoritma ini adalah kompleksitas waktunya linear, dipastikan selalu $O(n+m)$ pada semua kondisi. Algoritma ini juga tidak menerapkan *backtracking*, sehingga tidak pernah membaca karakter teks yang sama dua kali. Selain itu, KMP juga dapat bekerja dengan streaming text (*online algorithm*). Performansi dari algoritma ini juga dapat diprediksi, yaitu tidak ada degradasi performa pada input tertentu.

Akan tetapi, kekurangan dari KMP adalah *extra space*, di mana diperlukan ruang sebesar $O(m)$ untuk *failure function*. Selain itu, KMP juga perlu melakukan *preprocessing* untuk setiap *pattern*. KMP juga tidak optimal untuk *pattern* yang sangat panjang karena *random memory access (cache performance)*.

Aplikasi KMP yang optimal adalah untuk *searching* dalam *streaming data* atau file besar, *multiple searches* dengan *pattern* yang sama, sistem dengan memory constraint yang reasonable, serta *real-time text processing* yang memerlukan *predictable latency*.

2.2. Algoritma Boyer-Moore

Algoritma Boyer-Moore adalah algoritma pencocokan string yang dikembangkan oleh Robert S. Boyer dan J Strother Moore pada tahun 1977. Algoritma ini dikenal sebagai salah satu algoritma *string matching* praktis yang paling efisien, terutama untuk alfabet yang besar dan *pattern* yang panjang. Boyer-Moore menggunakan pendekatan yang berbeda dari algoritma konvensional dengan melakukan pencocokan dari kanan ke kiri (*right-to-left matching*) dalam *pattern*, namun tetap bergerak dari kiri ke kanan dalam teks. Pendekatan ini memungkinkan algoritma untuk "melompat" lebih jauh ketika terjadi ketidakcocokan.

Algoritma heuristik utama dari Boyer-Moore terdiri atas dua, yaitu heuristik karakter buruk dan heuristik *suffix* baik. Pada heuristik karakter buruk, ketika terjadi ketidakcocokan di karakter x dalam teks, jika x tidak muncul dalam *pattern*, geser *pattern* melewati x sepenuhnya. Jika x muncul dalam *pattern*, *align* kemunculan terakhir x dalam *pattern matching* dengan x di teks. Jika *alignment* menghasilkan pergeseran negatif atau nol, geser *pattern* satu posisi.

Di sisi lain, pada heuristik *suffix* baik memanfaatkan informasi dari *suffix* yang telah berhasil dicocokkan. Jika *suffix* S cocok tetapi karakter sebelumnya tidak cocok, dicari kemunculan S lain dalam *pattern*. Jika tidak ada kemunculan S lain, dicari *prefix pattern* yang merupakan *suffix* dari S . Boyer-Moore juga menerapkan *looking-glass technique*, yaitu *pattern* dibaca dari kanan ke kiri. Ketika ketidakcocokan terjadi, informasi digunakan untuk menentukan *skip distance*. *Skip distance* dapat lebih dari satu karakter, yang menghasilkan performa sublinear.

Kompleksitas algoritma Boyer-Moore terbagi menjadi tiga, yaitu sebagai berikut.

- Waktu Preprocessing : $O(m + \sigma)$ dimana σ adalah ukuran alfabet
- Waktu Pencarian
 1. Kasus Terbaik : $O(n/m)$, sublinear performance
 2. *Average Case* : $O(n)$ untuk teks natural dengan alphabet besar
 3. *Worst Case* : $O(nm)$, jarang terjadi dalam praktik
- Kompleksitas Ruang : $O(m + \sigma)$ untuk karakter buruk dan *suffix* tabel baik

Keunggulan algoritma BM adalah sublinear performance, dapat mencapai $O(n/m)$ pada kondisi ideal. Selain itu, algoritma ini juga sangat cepat untuk aplikasinya dalam dunia nyata. Algoritma ini juga adaptif dan memiliki performa yang lebih baik dengan *pattern* dan teks yang lebih sulit. Dengan *right to left scanning*, algoritma ini juga lebih *cache-efficient*.

Akan tetapi, algoritma ini lebih kompleks dan rumit dibandingkan KMP dan pada *worst case*-nya juga dapat degradasi ke $O(nm)$ pada input adversarial. Pencarian dengan algoritma ini kurang efisien pada alfabet kecil (seperti DNA: A, T, G, C). Selain itu, kekurangan terbesar dari algoritma ini adalah *overhead preprocessing*-nya yang sangat tidak sebanding untuk *pattern* pendek.

Algoritma Boyer-Moore juga memiliki beberapa variasi, yaitu Boyer-Moore-Horspool, Turbo Boyer-Moore, dan Boyer-Moore dengan Galil Rule. Boyer-Moore-Horspool adalah versi tersimplifikasinya yang hanya menggunakan heuristik karakter buruk. Kemudian, Turbo

Boyer-Moore adalah kombinasi Boyer-Moore dengan *shift optimization*. Terakhir, Boyer-Moore dengan Galil Rule adalah optimasi untuk menghindari perbandingan yang redundan.

Penerapan Boyer-Moore dapat ditemukan pada *text editors* dengan ukuran dokumen yang besar, pencarian dalam teks dengan bahasa natural, *grep* dan *text processing utilities*, *antivirus signature scanning*, *large-scale document indexing*, dan lainnya.

2.3. Algoritma Aho-Corasick

Algoritma Aho-Corasick adalah algoritma pencocokan string *multi-pattern* yang dikembangkan oleh Alfred Aho dan Margaret Corasick pada tahun 1975. Algoritma ini memungkinkan pencarian beberapa *pattern* secara simultan dalam satu kali traversal teks, menjadikannya ideal untuk aplikasi yang memerlukan deteksi kata kunci lebih dari satu.

Ketika perlu mencari banyak *pattern* dalam teks yang sama, pendekatan *brute-force* adalah menjalankan algoritma *single-pattern matching* untuk setiap *pattern* secara terpisah. Hal ini menghasilkan kompleksitas $O(n \times k)$ dimana k adalah jumlah *pattern*. Aho-Corasick mengatasi ketidakefisienan ini dengan memungkinkan pencarian semua *pattern* dalam sekali traversal dengan kompleksitas $O(n+m+z)$.

Struktur data yang fundamental dalam algoritma ini adalah *trie (prefix tree)*, *failure links (suffix links)*, dan *output links*. *Prefix tree* adalah struktur pohon yang menyimpan semua *pattern* dengan *sharing common prefixes*. Setiap *node* merepresentasikan *prefix* dari satu atau lebih *pattern*. *Leaf nodes* atau *internal nodes* dengan penanda menandakan akhir dari *pattern*. *Failure links (suffix links)* berfungsi sebagai *pointer* dari setiap *node* ke *node* lain yang merepresentasikan *longest proper suffix*, juga memungkinkan transisi yang efisien ketika tidak ada *matching child*. Selain itu, *failure links* ini juga berfungsi sebagai analog dengan *failure function* dalam KMP tetapi untuk struktur *tree*. *Output links* menghubungkan *node* ke semua *pattern* yang berakhir pada posisi tersebut dan memungkinkan deteksi *pattern* yang merupakan *suffix* dari *pattern* lain.

Tahapan dari algoritma Aho-Corasick terdiri dari tiga tahapan utama, yaitu *construction phase*, *failure function construction*, dan *search phase*. Pada *construction phase*, algoritma ini membangun *trie* dari semua *pattern* yang akan dicari. Kompleksitas dari fase ini adalah $O(m)$, dengan m sebagai total panjang semua *pattern*. Kemudian, pada fase *failure function construction*, fase ini membangun *failure links* menggunakan *Breadth First Search* dengan

kompleksitas $O(m \times \sigma)$ dengan σ adalah ukuran alfabet. Terakhir, pada *fase search*, traversal teks menggunakan trie dengan *failure links* dan kompleksitasnya adalah $O(n+z)$ dengan z sebagai jumlah *pattern occurrences*.

Failure link dari *node* u menunjuk ke *node* v yang merepresentasikan *longest proper suffix* dari string yang direpresentasikan oleh *path* dari *root* ke u . Ketika tidak ada *outgoing edge* untuk karakter tertentu, *failure link* digunakan untuk melakukan transisi yang tepat tanpa kehilangan *informasi matching*.

Adapun kompleksitas dari Aho-Corasick adalah sebagai berikut.

- *Preprocessing Time*: $O(m \times \sigma)$ untuk kasus terburuk, $O(m)$ untuk kasus rata-rata
- *Search Time*: $O(n+z)$ dimana z adalah jumlah kecocokan
- *Space Complexity*: $O(m \times \sigma)$ untuk struktur trie
- *Total Complexity*: $O(n+m+z)$ yang optimal untuk *multi-pattern matching*

Keunggulan dari algoritma ini adalah semua *pattern* dicari dalam satu traversal teks, kompleksitasnya tidak bergantung pada jumlah *pattern*, dan optimal untuk *multiple patterns*. Semakin banyak *pattern*, algoritma ini relatif semakin efisien. Selain itu, algoritma ini juga dapat mendeteksi *pattern* yang overlap. Algoritma ini dapat juga bekerja dengan streaming text (*streaming capability*).

Akan tetapi, algoritma Aho-Corasick memerlukan memori $O(m \times \sigma)$ yang besar untuk alfabet besar. Selain itu, terdapat juga *construction overhead* yang tinggi karena *preprocessing time*-nya signifikan untuk *pattern* yang sedikit. *Random memory access pattern* juga dapat mengurangi *cache efficiency*. Implementasi dari algoritma ini juga lebih kompleks dibanding *single-pattern algorithms*.

Untuk mengurangi dampak kekurangan dari algoritma ini, bisa juga dilakukan berbagai optimasi pada algoritma Aho-Corasick, seperti mengurangi *space complexity* dengan *path compression*, menggunakan *hash table* sebagai pengganti *array* untuk transisi, mengurangi jumlah *failure link traversals*, serta *mapping* karakter untuk mengurangi *effective alphabet size*.

Aplikasi dari algoritma ini dapat ditemukan dalam berbagai hal, seperti *network intrusion detection*, *antivirus scanning*, *text mining*, *content filtering*, *bioinformatics*, dan *log analysis*.

2.4 Algoritma Levenshtein Distance

Algoritma Levenshtein Distance, yang juga dikenal sebagai Edit Distance, adalah metrik untuk mengukur perbedaan antara dua string dengan menghitung jumlah minimum operasi

editing yang diperlukan untuk mengubah satu string menjadi string lainnya. Algoritma ini dikembangkan oleh Vladimir Levenshtein pada tahun 1965 dan menjadi dasar untuk berbagai aplikasi *approximate string matching*.

Operasi *edit* yang diizinkan pada algoritma ini adalah *insertion*, *deletion*, dan *substitution*. Pada operasi *insertion* (penyisipan), diperbolehkan untuk menambahkan satu karakter ke dalam string. Pada *deletion* (penghapusan), diperbolehkan untuk menghapus satu karakter dari string. Terakhir, pada *substitution* (penggantian), diperbolehkan untuk mengganti satu karakter dengan karakter lain.

Untuk dua string s_1 dengan panjang m dan s_2 dengan panjang n , Levenshtein Distance $lev(s_1, s_2)$ adalah jumlah minimum operasi edit yang diperlukan untuk mengubah s_1 menjadi s_2 . Levenshtein Distance dapat dihitung efisien menggunakan *dynamic programming* dengan membuat matriks berukuran $(m+1) \times (n+1)$. Setiap sel (i,j) dalam matriks menyimpan *edit distance* antara *prefix* $s_1[0..i-1]$ dan $s_2[0..j-1]$.

Adapun kompleksitas dari algoritma ini adalah sebagai berikut.

- Kompleksitas Waktu : $O(m \times n)$ untuk *standard dynamic programming*
- Kompleksitas Ruang : $O(m \times n)$ untuk menyimpan seluruh matriks, dapat dioptimasi menjadi $O(\min(m,n))$
- Optimasi Ruang : Perhitungan hanya memerlukan baris sebelumnya, jadi ruang dapat dikurangi

Kemudian, terdapat juga beberapa variasi dari algoritma Levenshtein ini, yaitu *weighted edit distance* (memberikan bobot berbeda untuk setiap operasi), Damerau-Levenshtein Distance (menambahkan operasi *transposition* (*swap adjacent characters*)), Longest Common Subsequence (LCS) (khusus untuk *insertion* dan *deletion* saja), dan Hamming Distance (khusus untuk *substitution* pada string dengan panjang sama).

Levenshtein Distance adalah algoritma yang digunakan untuk mengukur kemiripan antara dua string berdasarkan jumlah operasi minimum, seperti penyisipan, penghapusan, atau penggantian karakter, yang diperlukan untuk mengubah satu string menjadi string lainnya. Agar hasilnya lebih mudah dipahami, nilai jarak ini biasanya dinormalisasi menjadi persentase kemiripan dengan rumus: $\text{Similarity} = (1 - \text{Edit Distance} / \text{panjang maksimum kedua string}) \times 100\%$. Normalisasi ini penting agar perbandingan tetap adil meskipun panjang string berbeda. Dalam praktiknya, sistem sering menggunakan ambang batas (*threshold*) untuk menentukan

apakah dua string dianggap mirip. Ambang 90-100% bersifat sangat ketat dan hanya menerima kesalahan kecil, 70-90% masih cukup presisi dengan toleransi moderat, sedangkan 60-70% lebih longgar dan cocok untuk variasi penulisan. Di bawah 60%, risiko ketidakakuratannya meningkat.

Algoritma ini memiliki beberapa keunggulan, yaitu mudah dipahami, fleksibel digunakan di berbagai bahasa, dan banyak dikembangkan dengan berbagai optimasi. Akan tetapi, terdapat juga kelemahan, misalnya kebutuhan komputasi yang tinggi untuk string panjang (kompleksitas $O(mn)$), tidak mempertimbangkan konteks atau makna kata, dan memperlakukan semua jenis kesalahan secara setara. Selain itu, versi standar belum menangani pertukaran karakter yang berdampingan secara efisien, padahal itu merupakan jenis kesalahan yang umum. Untuk mengatasinya, dapat digunakan beberapa teknik optimasi seperti penghematan memori, penghentian proses lebih awal jika jarak terlalu jauh, serta fokus pada diagonal matriks jika string relatif mirip.

Penggunaan Levenshtein Distance sangat luas, seperti pada pemeriksaan ejaan, pencarian kata mendekati di database, deteksi plagiarisme, pembersihan data, pengenalan kata pada mesin pencari, serta sistem pelacakan pelamar kerja (ATS). Dalam ATS, algoritma ini digunakan untuk mencocokkan kata kunci meskipun terjadi salah ketik, seperti "Phyton" yang dikenali sebagai "Python", atau variasi penulisan seperti "color" dan "colour". Sistem juga dapat menggunakan ambang kemiripan (misalnya 60%) agar pencocokan tetap relevan. Sebagai contoh, masukan "machin learning" akan dikenali mirip dengan "machine learning" dengan skor kemiripan 85,7%, sehingga masih layak dianggap cocok dalam sistem pencarian yang toleran terhadap kesalahan.

2.5. Aplikasi yang Dibangun (Matchify)

Matchify adalah sebuah aplikasi yang dibangun untuk membantu proses pencarian dan pencocokan dokumen CV secara otomatis berdasarkan kata kunci tertentu. Aplikasi ini ditujukan untuk mempermudah sistem *Applicant Tracking System* (ATS) dalam mengidentifikasi kandidat yang paling relevan dari kumpulan CV yang besar.

Cara kerja aplikasi secara singkat adalah sebagai berikut:

1. Pengguna memasukkan satu atau lebih kata kunci terkait nama, posisi, atau keahlian yang dicari.
2. Pengguna memilih algoritma pencocokkan teks, seperti Knuth-Morris-Pratt (KMP), Boyer-Moore (BM), atau Aho-Corasick.

3. Pengguna memasukkan berapa banyak top CVs yang ingin ditampilkan.
4. Sistem akan memindai seluruh dokumen CV dalam database dan mencocokkan kata kunci tersebut terhadap isi dokumen.
5. Hasil pencarian ditampilkan secara interaktif dalam bentuk *card*, berisi nama pelamar, posisi yang dilamar, skor kecocokan, dan tombol untuk melihat ringkasan maupun file CV lengkap.
6. Pengguna dapat melihat detail pelamar secara lengkap melalui tampilan CV Summary View dan memilih cara untuk membuka file CV asli.

Aplikasi ini dibangun menggunakan Flet yaitu sebuah framework berbasis Python yang memungkinkan pengembangan aplikasi dan desktop dengan antarmuka yang interaktif. Flet mempermudah proses pembuatan UI karena semuanya ditulis dalam satu bahasa, yaitu Python. Dalam penyimpanan data, aplikasi ini menggunakan MySQL, yaitu sistem manajemen basis data relasional yang andal dan banyak digunakan. MySQL digunakan untuk menyimpan data pelamar, isi dokumen CV, dan hasil ekstraksi secara keseluruhan. Dengan kombinasi Flet dan MySQL, seluruh proses pencocokan CV mulai dari input hingga tampilan hasil dapat dijalankan secara efisien dalam satu ekosistem yang terintegrasi.

BAB 3

ANALISIS PEMECAHAN MASALAH

3.1. Langkah-Langkah Pemecahan Masalah

Masalah yang ada dalam Tugas Besar 3 ini adalah mencari pelamar kerja yang tepat berdasarkan kata kunci yang dimasukkan oleh pengguna dari kumpulan CV yang tersedia. Pencarian dilakukan dengan algoritma *string matching* yang ditentukan pengguna, antara lain Knuth-Morris-Pratt (KMP), Boyer-Moore (BM), atau Aho-Corasick (AC). Pengguna dapat memasukkan satu atau lebih kata kunci untuk pencarian. Jumlah hasil pencarian yang ditampilkan juga dapat disesuaikan dengan keinginan pengguna.

Pencarian pelamar kerja diawali dengan masukan dari pengguna berupa kata kunci terkait pelamar kerja atau lamaran kerja (CV), jenis algoritma pencarian, dan jumlah hasil pencarian yang ingin ditampilkan. Algoritma akan mengekstrak data dari file PDF yang tersedia dan mendeteksi isi PDF menggunakan *regex* agar masuk ke bagian CV yang tepat untuk ditampilkan. Kemudian, algoritma melakukan *string matching* antara isi CV dengan kata kunci masukan dan untuk setiap kecocokan yang ditemukan, CV dianggap semakin sesuai dengan kebutuhan pengguna. Apabila kata kunci tidak ditemukan di CV mana pun, akan dilakukan *string matching* dengan *fuzzy search* menggunakan algoritma Levenshtein yang mengizinkan kesalahan pengetikan atau *typo* untuk ambang batas tertentu.

Setelah pencarian selesai, CV yang paling sesuai dengan kata kunci masukan sejumlah jumlah hasil pencarian yang diinginkan pengguna ditampilkan. Setiap CV dapat diklik untuk melihat detailnya secara lebih lanjut yang ditampilkan dalam beberapa bagian untuk meningkatkan keterbacaan. File CV yang terkait juga dapat dibuka, baik dalam bentuk PDF maupun teks. Selain itu, ditampilkan waktu pencarian, jumlah file CV yang diproses, dan banyak ditemukannya kata kunci di seluruh CV.

3.2. Pemetaan Masalah Menjadi Elemen-Elemen Algoritma KMP

Dalam pemecahan masalah pencarian kata kunci pada file CV dengan algoritma Knuth-Morris-Pratt (KMP), permasalahan dipetakan dengan memodelkan dokumen sebagai

dokumen dipandang sebagai teks utama, sementara setiap kata kunci diperlakukan sebagai pola yang ingin dicocokkan. KMP memanfaatkan struktur data bantuan berupa tabel Longest Prefix Suffix (LPS) untuk menentukan sejauh mana pencarian dapat dilanjutkan tanpa perlu memeriksa ulang karakter sebelumnya. Dengan demikian, pencocokan dilakukan secara linear terhadap panjang teks dan pola, menghindari pencarian berulang yang tidak efisien.

3.3. Pemetaan Masalah Menjadi Elemen-Elemen Algoritma BM

Masalah pencarian kata kunci dalam file CV menggunakan algoritma Boyer-Moore (BM) dipetakan dengan menganggap dokumen sebagai teks utama yang akan dipindai, dan kata kunci sebagai pola pencarian. Algoritma ini mengoptimalkan pencarian dengan membandingkan pola dari belakang ke depan dan menggunakan strategi lompatan (shift) berdasarkan informasi karakter yang tidak cocok, melalui tabel Bad Character Heuristic. Tabel ini memetakan setiap karakter dalam pola ke posisi kemunculan terakhirnya, sehingga saat terjadi ketidaksesuaian, pola dapat digeser melewati bagian teks yang tidak mungkin cocok. Dengan teknik ini, algoritma Boyer-Moore mampu melewati sebagian besar karakter teks tanpa pemeriksaan satu per satu, terutama efektif saat pola relatif pendek dan teks panjang. Pemetaan ini menjadikan Boyer-Moore ideal untuk sistem pencarian kata kunci dalam dokumen berukuran besar, seperti pada aplikasi pemfilteran atau klasifikasi berbasis teks.

3.4. Pemetaan Masalah Menjadi Elemen-Elemen Algoritma Levenshtein

Dalam permasalahan pencarian berbasis teks, kesalahan penulisan (*typo*) atau variasi ejaan seringkali menjadi kendala dalam pencocokan kata kunci secara tepat. Untuk menangani hal ini, digunakan pendekatan pencocokan *fuzzy* dengan algoritma Levenshtein Distance. Dalam pemetaannya, setiap kata dalam teks diekstraksi sebagai unit perbandingan, sedangkan kata kunci menjadi pola acuan. Levenshtein Distance menghitung jumlah minimum operasi penyisipan, penghapusan, atau substitusi karakter yang diperlukan untuk mengubah satu kata menjadi kata lainnya. Dari nilai tersebut, dihitung persentase kemiripan untuk menentukan apakah suatu kata cukup dekat atau mirip dengan kata kunci. Sistem ini menggunakan ambang kemiripan sebesar 60% berdasarkan pengujian terhadap berbagai kasus *typo*, di mana pada ambang tersebut kata-kata yang seharusnya terdeteksi berhasil ditemukan tanpa menghasilkan

kata lain yang tidak relevan. Dengan demikian, pemetaan ini memungkinkan pencarian yang lebih toleran terhadap variasi input pengguna, sekaligus menjaga akurasi hasil yang ditampilkan.

3.5. Pemetaan Masalah Menjadi Elemen-Elemen Algoritma Aho-Corasick

Untuk menyelesaikan masalah pencarian kata kunci di dokumen dengan algoritma Aho-Corasick, masalah dipetakan ke dalam elemen-elemen struktur pohon trie, di mana setiap karakter kata kunci menjadi simpul. Setelah seluruh pola dimasukkan, dibentuk fungsi failure link untuk mengatur transisi saat terjadi ketidakcocokan karakter. Proses pencarian dilakukan dalam satu lintasan linear terhadap teks, sambil menavigasi simpul trie dan mengikuti failure link saat perlu. Ketika simpul yang dilewati mengandung output (kata kunci ditemukan), posisi kemunculannya dicatat. Dengan pemetaan ini, sistem dapat menemukan semua kata kunci sekaligus secara optimal, tanpa harus melakukan pencarian berulang untuk setiap kata kunci seperti pada algoritma KMP.

3.6. Fitur Fungsional

Aplikasi ini memiliki beberapa fitur utama yang saling mendukung dalam proses pencarian dan analisis dokumen CV. Seluruh fitur dirancang untuk bekerja secara terintegrasi dalam sistem berbasis Python, yang memanfaatkan algoritma pencocokan string serta manajemen data yang efisien.

a. Ekstraksi

Fitur ini bertanggung jawab untuk membaca dan mengekstrak isi teks dari file PDF CV menggunakan pustaka seperti PyPDF2 atau fitz. Hasil ekstraksi disimpan dalam bentuk teks mentah dan dipecah menjadi beberapa bagian penting seperti summary, skills, experience, education, dan accomplishment menggunakan regex.

b. Algoritma Knuth-Morris-Pratt (KMP)

Algoritma KMP diimplementasikan dengan melakukan pencarian pola (keyword) dalam teks dengan preprocessing array Longest Prefix Suffix (LPS) untuk menghindari pengulangan pemeriksaan karakter.

c. Algoritma Boyer-Moore (BM)

Algoritma BM diimplementasikan dengan menggunakan strategi *bad character rule* untuk melakukan lompatan besar saat pola tidak cocok yang membuatnya efisien untuk teks panjang.

d. Algoritma Aho-Corasick

Algoritma Aho-Corasick merupakan algoritma pencarian multiple keyword yang diimplementasikan dalam class `AhoCorasick`. Algoritma ini dibangun menggunakan struktur data trie dan *failure function* untuk mempercepat pencarian banyak kata sekaligus. Algoritma ini cocok digunakan saat pengguna memasukkan lebih dari satu kata kunci.

e. Algoritma Levenshtein Distance (Fuzzy Matching)

Algoritma Levenshtein digunakan untuk fitur fuzzy search, yaitu pencarian berdasarkan kemiripan kata. Algoritma ini menghitung tingkat kesamaan dua kata dalam bentuk persentase. Dalam implementasinya, terdapat fungsi yang digunakan untuk mencari kata-kata dalam CV yang mirip dengan kata kunci berdasarkan ambang batas (threshold).

3.7. Arsitektur Aplikasi

a. Backend

Bagian backend dari aplikasi ini dibangun menggunakan Python dan terdiri dari beberapa komponen modular yang dipisahkan dalam file yang berbeda, seperti file untuk algoritma KMP, Boyer-Moore, Aho-Corasick, serta Levenshtein Distance. Backend menangani seluruh logika pencarian, manajemen koneksi database, dan pengelolaan hasil pencarian.

File `database.py` bertanggung jawab atas koneksi dan interaksi dengan database MySQL. File ini juga menangani pembacaan, penulisan, dan dekripsi data dari tabel `ApplicantProfile` dan `ApplicationDetail`. File `search_engine.py` bertanggung

jawab untuk memuat CV dan data pelamar dari database, lalu menjalankan proses pencarian keyword menggunakan algoritma yang dipilih. Hasil pencarian kemudian diurutkan berdasarkan skor kemiripan.

b. Frontend

Antarmuka aplikasi dibangun dengan menggunakan Flet yaitu sebuah framework berbasis Python untuk pengembangan aplikasi desktop, dan mobile secara efisien. Fitur-fitur yang diimplementasikan di antarmuka, antara lain input keywords, select algorithm, top CVs selector, search button, hasil pencarian, view summary, dan view CV.

Struktur frontend dipisahkan ke dalam dua file utama, yaitu views.py yang bertanggung jawab untuk membangun dan menampilkan seluruh elemen UI, seperti input field, tombol, dan hasil pencarian. File kedua adalah controller.py yang berfungsi sebagai penghubung antara frontend dan backend. File disini menangani event seperti klik tombol Search, lalu memanggil fungsi pencarian dari backend dan mengatur data yang akan ditampilkan di frontend.

c. Database

Sistem ini menggunakan MySQL sebagai basis data relasional untuk menyimpan informasi pelamar dan detail dokumen CV yang telah diproses. Database ini terdiri dari dua tabel, yaitu ApplicantProfile dan ApplicationDetail. Tabel ApplicantProfile adalah tabel yang menyimpan data profil dasar pelamar seperti nama, tanggal lahir, alamat, dan nomor telepon. Tabel ApplicationDetail adalah tabel yang menyimpan detail setiap CV yang diunggah, termasuk teks hasil ekstraksi dan ringkasan bagian-bagian penting dalam dokumen, seperti Summary, Skill, Experience, Education, dan Accomplishment.

Pada database ini, file PDF CV diproses menggunakan PyPDF2 lalu diidentifikasi bagian-bagian penting seperti skill, education, dan sebagainya, lalu disimpan. Sistem ini juga menyediakan fitur ekspor seluruh isi database ke file .sql agar mudah di-*backup*.

3.8. Contoh Ilustrasi Kasus

- Kasus

Seorang user ingin mencari pelamar yang memiliki keahlian python dan machine learning dari kumpulan CV yang tersedia dalam SQL.

- Langkah Pemrosesan

1. Pengguna memasukkan dua kata kunci, yaitu “python, machine learning”
2. Pengguna memilih algoritma pencocokan, misalnya Knuth-Morris-Pratt (KMP)
3. Pengguna menentukan batas hasil sebanyak top 10 CVs
4. Sistem melakukan proses pencarian sebagai berikut:
 - Sistem menggunakan KMP untuk mencocokkan keyword dengan teks dalam CV dan profil pelamar.
 - Sistem menghitung jumlah kemunculan masing-masing keyword.
 - Sistem mengurutkan hasil berdasarkan skor total kecocokan.
 - Jika tidak ditemukan kecocokan exact, sistem akan menjalankan pencarian fuzzy menggunakan Levenshtein Distance secara otomatis.

- Output Sistem

1. Sistem akan menampilkan daftar CV dengan skor pencocokan tertinggi.
2. Sistem menampilkan statistik pencarian yang berisi informasi algoritma yang dipakai, waktu eksekusi, jumlah dokumen yang dipindai, dan jumlah total kemunculan kata.
3. Pengguna juga dapat melihat fitur lebih lanjut untuk menampilkan summary CV dan membuka file CV asli dari salah satu pelamar pada hasil pencarian.

BAB 4

IMPLEMENTASI DAN PENGUJIAN

4.1. Spesifikasi Teknis Program Database

4.1.1. Struktur Data

Struktur Data	Penjelasan
ApplicantProfile	Tabel untuk menyimpan data pelamar, seperti nama, tanggal lahir, alamat, nomor telepon, dan flag is_encrypted
ApplicationDetail	Tabel untuk menyimpan detail aplikasi tiap pelamar, seperti peran yang dilamar, dan path file CV
sections	Dictionary yang berisi hasil ekstraksi dari CV berdasarkan bagian-bagian seperti summary, skills, experience, education, dan accomplishment

4.1.2. Fungsi dan Prosedur

Fungsi atau Prosedur	Penjelasan
create_applicant_profile_table	Perintah SQL untuk membuat tabel ApplicantProfile jika belum ada
create_application_detail_table	Perintah SQL untuk membuat tabel ApplicationDetail jika belum ada
extract_text_from_pdf(path)	Mengekstrak teks dari file PDF menggunakan PyPDF2
extract_cv_sections(text)	Mengambil bagian penting dari isi CV seperti summary, skills, experiences, dan sebagainya, dengan regex dan strategi fallback
insert_applicant_profile(profile_data, encrypt)	Menyimpan data pelamar ke tabel ApplicantProfile
insert_applicant_detail(applicant_id, path, text, sections, encrypt)	Menyimpan detail CV yang sudah diproses ke tabel ApplicationDetail

process_folder(folder_path, encrypt)	Memproses semua file PDF dalam suatu folder serta mengekstrak isi dan menyimpannya ke database
export_data_to_sql(filename)	Mengekspor seluruh data di database ke dalam file .sql lengkap dengan struktur tabel dan data serta dapat digunakan untuk backup atau migrasi data

4.2. Spesifikasi Teknis Program Backend

4.2.1. Struktur Data

Struktur Data	Penjelasan
ApplicantProfile	Kelas data yang merepresentasikan satu pelamar, mencakup nama, tanggal lahir, alamat, nomor HP
ApplicationDetail	Kelas data yang merepresentasikan detail aplikasi CV dari seorang pelamar,
DatabaseManager	Kelas utama yang menangani koneksi ke database, eksekusi query, pengambilan data
SearchAlgorithm	Enum yang mendefinisikan tiga pilihan algoritma pencarian, yaitu KMP, BOYER_MOORE, dan AHO_CORASICK
CVMatch	Kelas data untuk menyimpan hasil pencarian CV individual, termasuk total match, rincian keyword, dan sumber pencocokan
SearchResult	Kelas data untuk menyimpan hasil pencarian lengkap, termasuk statistik waktu, jumlah pencarian, hasil fuzzy, dan daftar CVMatch
DatabaseCVSearchEngine	Kelas utama yang menangani seluruh proses pencarian berbasis database, termasuk memuat cache, pencocokan keyword, fuzzy match, dan ranking
TrieNode	Node struktur data trie untuk membangun

	automata pada algoritma Aho-Corasick
AhoCorasick	Kelas yang mengimplementasikan algoritma pencarian string banyak pola sekaligus dengan preprocessing menggunakan failure function

4.2.2. Fungsi dan Prosedur

Fungsi atau Prosedur	Penjelasan
execute_query(query, params)	Menjalankan query SQL terhadap database dengan parameter opsional
get_all_applications()	Mengambil seluruh data aplikasi dari tabel ApplicationDetail
get_application_by_id(detail_id)	Mengambil satu detail aplikasi berdasarkan ID tertentu
search_applications_by_text(search, term)	Melakukan pencarian CV berdasarkan kata yang dicari dalam teks isi dan bagian penting CV
get_cv_texts_for_search()	Mengambil semua isi teks CV dari database untuk digunakan dalam proses pencocokan dengan algoritma pencarian string
compute_lps(pattern)	Menghitung array Longest Prefix Suffix (LPS) sebagai tahap awal dalam algoritma KMP
kmp_search(text, pattern)	Mencari semua posisi kemunculan pattern dalam text menggunakan algoritma KMP
compute_bad_char_table(pattern)	Membuat tabel pergeseran karakter untuk algoritma BM
boyer_moore_search(text, pattern)	Melakukan pencarian semua patterns dalam text secara bersamaan menggunakan algoritma Aho-Corasick
levenshtein_distance(s1, s2)	Menghitung jarak edit antar dua string (s1 dan s2) menggunakan metode Levenshtein Distance

similarity_percentage(s1, s2)	Menghitung jarak edit antar dua string (s1, s2) menggunakan metode Levensthein Distance
fuzzy_search_keywords(text, keywords)	Mencari kata dalam text yang miri[dengan keywords berdasarkan ambang batas kemiripan (threshold)
parse_keywords(input_str)	Memecah input string menjadi daftar keyword
count_keyword_occurences_in_file(filepath, keyword_string, threshold)	Mengembalikan daftar kata dalam file yang memiliki kemiripan dengan setiap keyword berdasarkan threshold
load_cv_cache()	Memuat teks CV dari database dan menyimpannya dalam cache
load_applicant_cache()	Memuat data profil pelamar untuk pencarian berbasis nama dan peran yang dilamar
search_cvs()	Fungsi utama pencarian yang menggabungkan pencocokan exact dan fuzzy match di CV maupun di profil pelamar
_perform_exact_search_on_cvs()	Melakukan pencocokan exact keyword dalam isi CV menggunakan algoritma yang dipilih
_perform_exact_search_on_profiles()	Melakuka pencocokan keyword dalam nama dan peran pelamar
_perform_fuzzy_search_on_cvs()	Melakukan pencarian fuzzy match dalam isi CV berdasarkan persentase kemiripan
_perform_fuzzy_search_on_profiles()	Melakukan pencarian fuzzy keyword dalam profil pelamar
_rank_cvs()	Menyusun peringkat hasil pencarian berdasarkan skor dan pencocokan exact dan fuzzy serta mengambil data profil dan CV untuk ditampilkan
get_cv_summary(detail_id)	Mengambil data ringkasan dari satu CV berdasarkan detail_id

4.3. Spesifikasi Teknis Program Frontend

4.3.1. Struktur Data

Struktur Data	Penjelasan
ats_controller	Objek dari kelas ATSController yang digunakan sebagai jembatan antara frontend dan backend
last_search_results	List hasil pencarian CV dalam bentuk dictionary yang memuat data pelamar, skor, dan path file
last_search_metadata	Dictionary berisi metadata pencarian, seperti algoritma yang digunakan, waktu eksekusi, dan total matches
pdf_info	Dictionary informasi file PDF, seperti nama file, ukuran, dan path
summary	Dictionary berisi detail CV pelamar, seperti nama, role, kontak, pengalaman, skill, dan lainnya yang diambil dari backend dan ditampilkan di halaman ringkasan pelamar

4.3.2. Fungsi dan Prosedur

Fungsi atau Prosedur	Penjelasan
on_search(e)	Fungsi utama untuk melakukan pemanggilan fungsi pencarian ke backend dari input pengguna lalu menampilkan hasilnya
render_paginated_results()	Menampilkan hasil pencarian dalam bentuk kartu CV dengan sistem <i>pagination</i> setiap 5 cards serta tombol navigasi
show_summary(detail_id)	Menampilkan ringkasan (summary) dari satu pelamar berdasarkan detail_id yang diklik
show_pdf_dialog(page, path, name)	Menampilkan dialog interaktif untuk membuka file CV asli
show_pdf_as_text(page, path)	Mengekstrak isi PDF menjadi teks biasa lalu menampilkannya dalam bentuk dialog

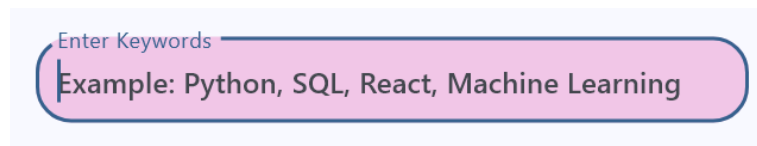
<code>open_pdf_with_system_viewer(path)</code>	Membuka file PDF menggunakan PDF viewer default sistem operasi
<code>copy_path_and_close(e)</code>	Menyalin path file PDF ke clipboard pengguna
<code>create_summary_section(tittle, content)</code>	Membuat bagian collapsible pada halaman summary agar dapat dibaca dalam mode “read more” dan “show less”
<code>render_search_metadata()</code>	Menampilkan informasi statistik dari hasil pencarian, seperti algoritma yang digunakan, waktu eksekusi, dan total matches
<code>build_layout()</code>	Membangun seluruh struktur tampilan halaman utama, termasuk navbar, input, keyword, pengaturan pencarian, tombol search, dan area hasil

4.4. Tata Cara Penggunaan Program

4.4.1. Fitur-Fitur Frontend

Aplikasi ini menyediakan beberapa fitur utama yang memungkinkan pengguna untuk mencari dan memfilter daftar CV berdasarkan kata kunci dan algoritma tertentu. Pengguna juga dapat mengatur parameter banyaknya CV yang ingin ditampilkan sesuai preferensi melalui antarmuka yang interaktif dan intuitif. Fitur-fitur ini membantu proses pencocokan CV menjadi lebih cepat dan efisien.

a. Enter Keywords



Fitur ini berfungsi untuk menerima masukan berupa kata kunci dari pengguna. Kata kunci yang dimasukkan akan digunakan sebagai dasar pencarian untuk mencocokkan isi dari CV yang tersedia dalam basis data. Sistem akan melakukan

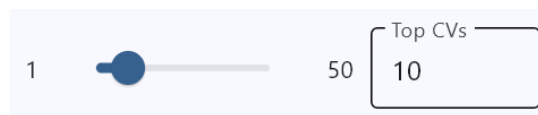
pencarian terhadap dokumen CV yang mengandung kata kunci tersebut sesuai dengan algoritma pencarian yang dipilih oleh pengguna.

b. Select Algorithm



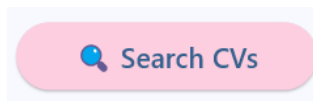
Fitur ini berfungsi untuk memungkinkan pengguna memilih algoritma pencocokan string yang akan digunakan dalam proses pencarian CV. Algoritma yang tersedia pada aplikasi kami diantaranya adalah Knuth-Morris-Pratt (KMP), Boyer-Moore (BM), dan Aho-Corasick.

c. Input Top CVs



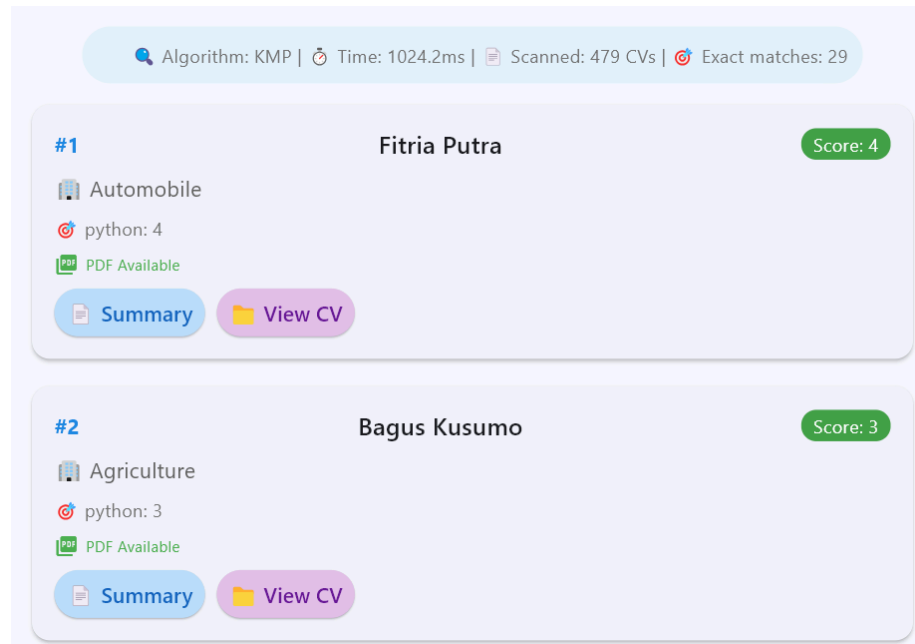
Fitur ini memungkinkan pengguna menentukan jumlah hasil pencarian teratas yang ingin ditampilkan. Pengguna dapat memasukkan angka secara manual melalui input box atau mengatur nilai menggunakan slider interaktif.

d. Tombol Search



Fitur ini berupa tombol yang digunakan untuk memulai proses pencarian. Setelah pengguna mengisi kata kunci, memilih algoritma, dan menentukan jumlah hasil teratas, pengguna dapat menekan tombol ini untuk menjalankan pencarian.

e. Hasil Query



Fitur ini berfungsi untuk menampilkan hasil pencarian CV berdasarkan kata kunci yang telah dimasukkan dan algoritma yang dipilih. Terdapat informasi statistik yang akan ditampilkan yang berisi algoritma yang dipilih, waktu eksekusi, serta jumlah CV yang dipindai, jumlah exact matches atau total kecocokan kata kunci yang ditemukan secara tepat dalam semua CV yang dipindai. Jika tidak ada exact match yang ditemukan, maka program akan otomatis mencari dengan fuzzy match dan akan ditampilkan juga lama waktu eksekusi menggunakan fuzzy match.

Dibawah informasi statistik, terdapat hasil pencarian yang ditampilkan dalam bentuk card yang diurutkan berdasarkan relevansi atau jumlah kecocokan kata kunci (score). Setiap card berisi informasi nama pelamar, application role, skor pencocokan, serta tombol aksi summary dan view CV.

f. View Summary


Cakrawangsa Wacana

 085286382248

Business Development

 Gg. Pacuan Kuda No. 722, Bima, JK 88430

  12546838.pdf

 **Summary**

With 11 years of diverse managerial and marketing experience penetrating in domestic and international markets including Manufacturing, Engineering, Food and IT, I have honed my communication and marketing research abilities to gauge demand for products and explore new clients. Having managed severa...

[Read more](#)

 **Skills**

acquisitions, Adobe Photoshop, advertising, approach, Automobiles, Basic, Brochures, budget, charts, closing, color, concept, conferences, Corel Draw, client, clients, Customer service, Databases, Decision Making, delivery, designing, Editing, ERP, Senior Management, financial, functional, graphs, h...

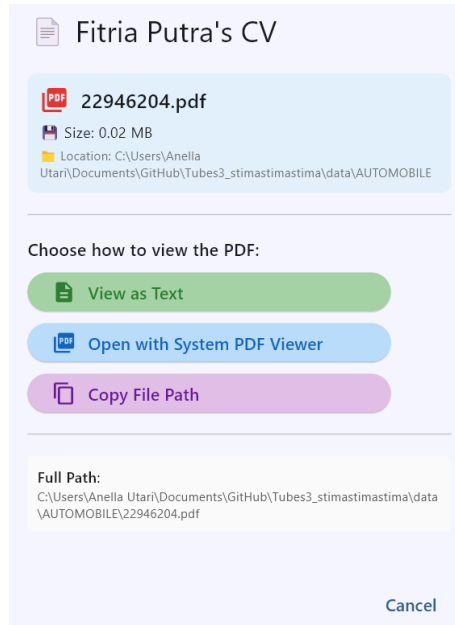
[Read more](#)

 **Experience**

Business Development Manager
February 2015 to May 2017 Company Name 1/4 City
Responsibilities in Project 1:

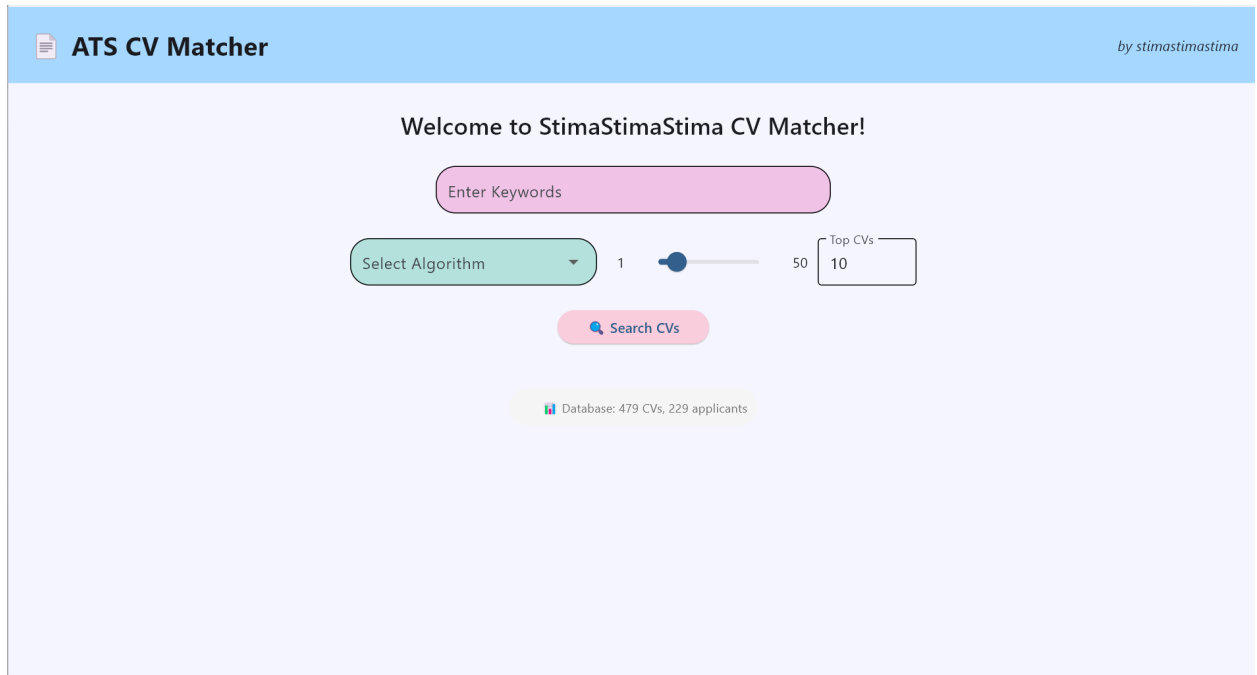
Fitur ini memberikan tampilan detail dari satu CV pelamar dalam format yang terstruktur dan mudah dibaca. Informasi yang ditampilkan meliputi nama pelamar, nomor telepon, alamat lengkap, serta beberapa section mengenai informasi penting pada CV, seperti Summary, Skills, Experience, Education, dan Accomplishment.

g. View CV



Fitur ini memungkinkan pengguna untuk melihat isi lengkap dari CV pelamar yang ditampilkan pada hasil pencarian. Setelah pengguna menekan tombol View CV, akan muncul tampilan baru yang menampilkan informasi berkas dan tiga opsi interaktif untuk melihat dokumen CV. Opsi pertama adalah View as Text, yaitu menampilkan isi file PDF dalam bentuk teks biasa melalui proses ekstraksi teks dari pdf. Opsi kedua adalah Open with System PDF Viewer, yaitu membuka file PDF menggunakan aplikasi penampil PDF bawaan sistem operasi pengguna. Opsi ketiga adalah Copy File Path, yaitu menyalin lokasi lengkap file PDF pada sistem file lokal pengguna ke clipboard.

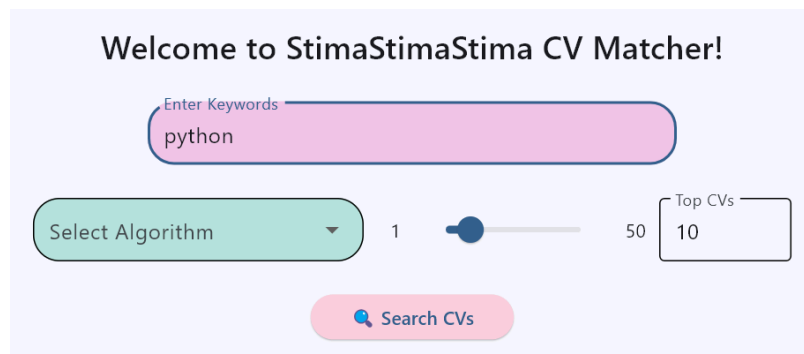
4.4.2. Search Interface



The screenshot shows the 'ATS CV Matcher' web application. At the top, there's a blue header with the title 'ATS CV Matcher' and the author 'by stimastimastima'. The main content area has a light purple background. It starts with a welcome message 'Welcome to StimaStimaStima CV Matcher!'. Below this is a search form with three main components: a pink input field labeled 'Enter Keywords', a green dropdown menu labeled 'Select Algorithm', and a slider control with a blue knob. To the right of the slider is a 'Top CVs' input field with the value '10'. A pink 'Search CVs' button is positioned below the input fields. At the bottom, a small status bar indicates 'Database: 479 CVs, 229 applicants'.

Pada tampilan awal aplikasi ini, terdapat box-box untuk memasukkan input keywords, memilih algoritma, dan menentukan berapa banyak CV teratas yang ingin ditampilkan. Berikut merupakan contoh cara mencari keywords “python” dengan menggunakan algoritma KMP dan ingin menampilkan 20 top CVs.

1. Masukkan keyword yang akan dicari



This screenshot shows the same search interface as the previous one, but with the keyword 'python' entered into the 'Enter Keywords' field. The 'Select Algorithm' dropdown is still set to '1', the slider is at the '1' position, and the 'Top CVs' field still shows '10'. The 'Search CVs' button remains visible.

2. Pilih algoritma yang ingin digunakan dengan menekan dropdown *select algorithm*

Welcome to StimaStimaStima CV Matcher!

Enter Keywords

KMP (Knuth-Morris-Pratt)
 BM (Boyer-Moore)
 AC (Aho-Corasick)

1

Top CVs

Search CVs

Database: 479 CVs, 229 applicants

3. Tentukan banyaknya top CVs yang ingin ditampilkan dengan menggeser slider atau langsung mengetikkannya di box Top CVs

Welcome to StimaStimaStima CV Matcher!

Enter Keywords

Select Algorithm
 KMP (Knuth-Morris-Pratt)

1

Top CVs

Search CVs

4. Tekan tombol “Search CVs” lalu tunggu hingga hasil pencarian keluar

ATS CV Matcher by stimastimastima

Welcome to StimaStimaStima CV Matcher!

Enter Keywords

Select Algorithm
 KMP (Knuth-Morris-Pratt)

1

Top CVs

Search CVs

Searching CVs...

5. Hasil pencarian akan ditampilkan berupa informasi algoritma yang dipakai, waktu eksekusi, total kata yang ditemukan dengan *exact match*, dan list CV yang diurutkan dari *exact matches* terbanyak dalam bentuk *pagination*.

Algorithm: KMP Time: 1016.9ms Scanned: 479 CVs Exact matches: 29		
#1	Wakiman Namaga	Score: 4
Automobile		
python: 4		
PDF Available		
Summary View CV		
#2	Muni Saefullah	Score: 3
Agriculture		
python: 3		
PDF Available		
Summary View CV		
#3	Cager Wibisono	Score: 3
Automobile		
python: 3		
PDF Available		
Summary View CV		
#4	Cakrawangsa Wacana	Score: 3

4.4.3. CV Summary View

Setelah aplikasi menampilkan hasil pencarian, pada setiap list CV akan terdapat button Summary yang akan membawa pengguna ke tampilan CV Summary. Pada halaman ini, akan terdapat ringkasan lengkap dari CV pelamar tersebut, termasuk informasi pribadi (nama, application role, nomor telepon, alamat rumah), pengalaman kerja, pendidikan, hingga opsi untuk melihat file CV asli.

ATS CV Matcher

by stimastimastima

CV Summary

Wakiman Namaga

087951600629

Automobile

Jalan K.H. Wahid Hasyim No. 5, Pekalongan, Jawa Timur 73308

22946204.pdf

Summary

Not available

Skills

3 D, assembly language, Auto CAD, automobiles, C, C language, CAD, Chinese, Clustering, com, controller, data analysis, Database, database software, designing, economics, Editing, embedded system, English, Experiments, GUI, graphic, Java, Laser, Lathe, Machine Learning, MATLAB, Mechanical, Mechanica...

Read more

Experience

Company Name June 2016 to Current R&D Product Development Engineer
City
Design and build a tail-sitter VTOL(vertical take off and landing) UAV(unmanned aerial vehicle) which takes off and lands vertically and travels horizontally.
Main duties include but not limit to aerodynamics modeling, UAV con...

Read more

Education

University of Pennsylvania, School of Eng. and Applied Science Aug, 2014 Master of Science : Mech. Eng. & Applied Mechanics Jun City , State
Mech. Eng. & Applied Mechanics Jun
Harbin Institute of Technology (HIT) 2010 Bachelor of Science : Mechanical Design and Automation Sep Aug City , China Mechan...

Read more

Accomplishments

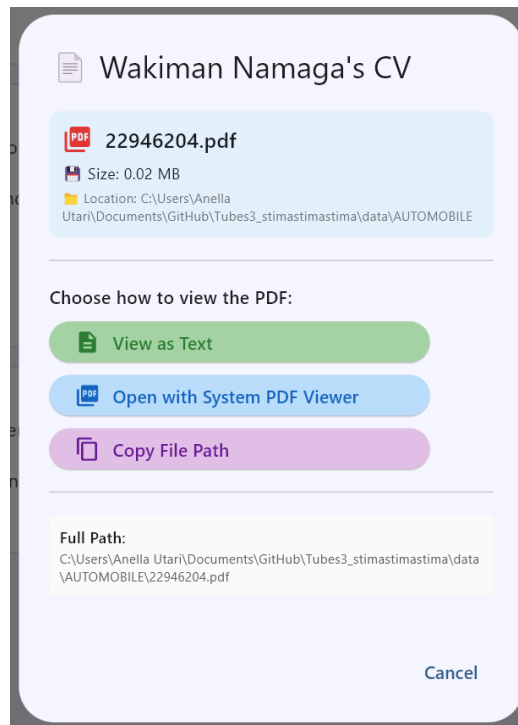
Not available

View CV

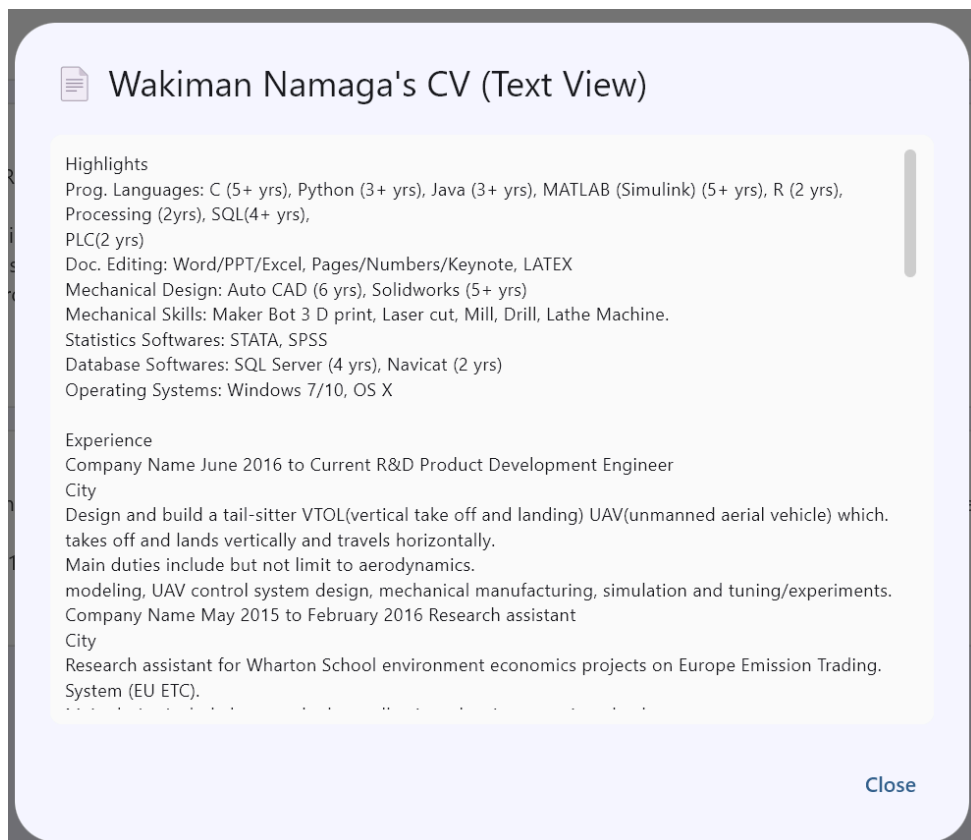
Back

Di bawah tampilan view summary, terdapat tombol Back dan View CV. Tombol Back adalah tombol untuk kembali kepada halaman awal aplikasi yang menampilkan hasil pencarian. Tombol View CV adalah tombol untuk menampilkan dokumen CV secara lengkap dengan beberapa cara. Jika pengguna menekan tombol View CV, akan ditampilkan dialog yang berisi informasi file CV (nama, ukuran, dan path file), serta button pilihan membuka file CV. Terdapat 3 button, yaitu View as Text, Open with System PDF Viewer, dan Copy File Path.

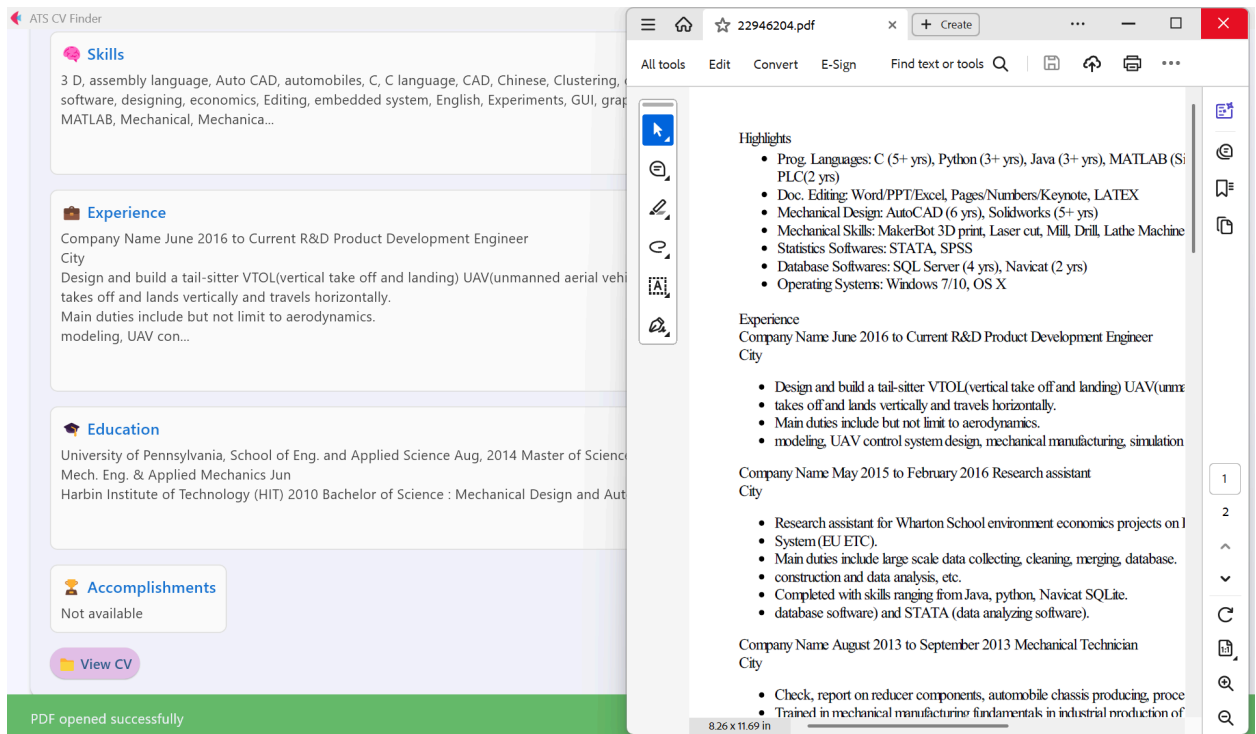
1. Tampilan saat klik button View CV



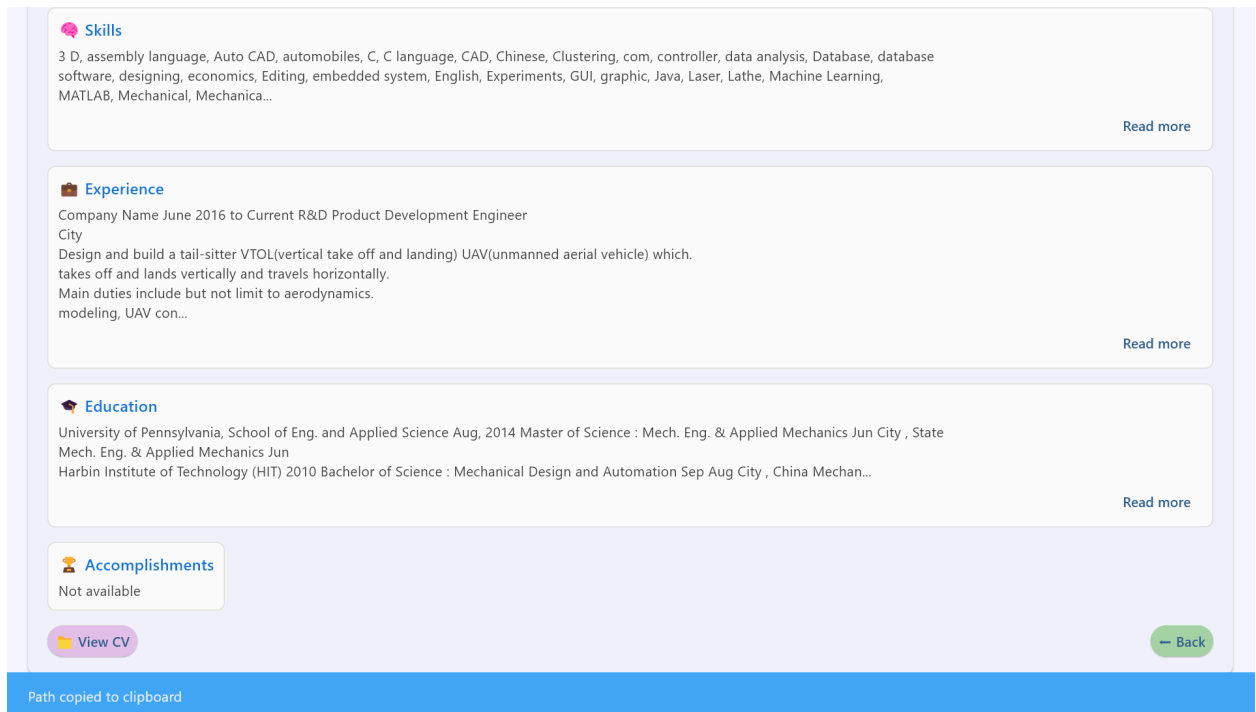
2. Tampilan saat klik View as Text



3. Tampilan saat klik Open with System PDF Viewer



4. Tampilan saat klik Copy File Path



4.5. Pengujian

4.5.1. Pengujian 1

- Menggunakan keyword “sql” dengan Algoritma KMP

ATS CV Matcher by stimastimastima

Welcome to Matchify - ATS CV Matcher!

Enter Keywords:

Select Algorithm: **KMP (Knuth-Morris-Pratt)** | 1 | 50 | Top CVs:

[Search CVs](#)

Algorithm: KMP | Time: 1228.8ms | Scanned: 592 CVs | Exact matches: 180

#1 **RaDen FrAnC1SCo** Score: 14

Cloud Architect

sql: 14

PDF Available

[Summary](#) [View CV](#)

#2 **RaDen FrAnC1SCo** Score: 14

- Menggunakan keyword “sql” dengan Algoritma BM

ATS CV Matcher by stimastimastima

Welcome to Matchify - ATS CV Matcher!

Enter Keywords:

Select Algorithm: **BM (Boyer-Moore)** | 1 | 50 | Top CVs:

[Search CVs](#)

Algorithm: BOYER_MOORE | Time: 818.8ms | Scanned: 592 CVs | Exact matches: 180

#1 **RaDen FrAnC1SCo** Score: 14

Cloud Architect

sql: 14

PDF Available

[Summary](#) [View CV](#)

#2 **RaDen FrAnC1SCo** Score: 14

- Menggunakan keyword “sql” dengan Algoritma Aho-Corasick

ATS CV Matcher by stimastimastima

Welcome to Matchify - ATS CV Matcher!

Enter Keywords:

Select Algorithm: AC (Aho-Corasick) 1 50 Top CVs: 10

[Search CVs](#)

Algorithm: AHO_CORASICK | Time: 796.8ms | Scanned: 592 CVs | Exact matches: 180

#1 RaDen FrAnC1SCo Score: 14

Cloud Architect

sql: 14

PDF Available

[Summary](#) [View CV](#)

#2 RaDen FrAnC1SCo Score: 14

4.5.2. Pengujian 2

- Menggunakan kata “administration” dengan Algoritma KMP

ATS CV Matcher by stimastimastima

Welcome to Matchify - ATS CV Matcher!

Enter Keywords:

Select Algorithm: KMP (Knuth-Morris-Pratt) 1 50 Top CVs: 10

[Search CVs](#)

Algorithm: KMP | Time: 1329.5ms | Scanned: 592 CVs | Exact matches: 132

#1 1khwan 4lh4k1m Score: 21

DevOps Engineer

administration: 21

PDF Available

[Summary](#) [View CV](#)

#2 MoH4mM4d NugR4h4 Score: 10

- Menggunakan kata “administration” dengan Algoritma BM

ATS CV Matcher by stimastimastima

Welcome to Matchify - ATS CV Matcher!

Enter Keywords
administration

Select Algorithm
BM (Boyer-Moore)

1 50 Top CVs
10

Search CVs

Algorithm: BOYER_MOORE | Time: 244.6ms | Scanned: 592 CVs | Exact matches: 132

#1 1khwan 4lh4k1m Score: 21

DevOps Engineer
administration: 21
PDF Available
Summary View CV

#2 MoH4mM4d NugR4h4 Score: 10

- Menggunakan kata “administration” dengan Algoritma Aho-Corasick

ATS CV Matcher by stimastimastima

Welcome to Matchify - ATS CV Matcher!

Enter Keywords
administration

Select Algorithm
AC (Aho-Corasick)

1 50 Top CVs
10

Search CVs

Algorithm: AHO_CORASICK | Time: 781.2ms | Scanned: 592 CVs | Exact matches: 132

#1 1khwan 4lh4k1m Score: 21

DevOps Engineer
administration: 21
PDF Available
Summary View CV

#2 MoH4mM4d NugR4h4 Score: 10

4.5.3. Pengujian 3

- Menggunakan keyword “python, react” dengan Algoritma KMP

ATS CV Matcher by stimastimastima

Welcome to Matchify - ATS CV Matcher!

Enter Keywords
python, react

Select Algorithm
KMP (Knuth-Morris-Pratt)

1 50 Top CVs
10

Search CVs

Algorithm: KMP | Time: 2447.7ms | Scanned: 592 CVs | Exact matches: 47

#1 1khwan 4lh4k1m Score: 4

Quality Control Inspector
python: 4
PDF Available
Summary View CV

#2 H41k4l 455y4uq1 Score: 4

- Menggunakan keyword “python, react” dengan Algoritma BM

ATS CV Matcher by stimastimastima

Welcome to Matchify - ATS CV Matcher!

Enter Keywords
python, react

Select Algorithm
BM (Boyer-Moore)

1 50 Top CVs
10

Search CVs

Algorithm: BOYER_MOORE | Time: 903.2ms | Scanned: 592 CVs | Exact matches: 47

#1 1khwan 4lh4k1m Score: 4

Quality Control Inspector
python: 4
PDF Available
Summary View CV

#2 H41k4l 455y4uq1 Score: 4

- Menggunakan keyword “python, react” dengan Algoritma Aho-Corasick

ATS CV Matcher

by stimastimastima

Welcome to Matchify - ATS CV Matcher!

Enter Keywords

python, react

Select Algorithm

AC (Aho-Corasick)

1

50

Top CVs

10

Search CVs

Algorithm: AHO_CORASICK | Time: 1353.6ms | Scanned: 592 CVs | Exact matches: 47

#1

1khwan 4lh4k1m

Score: 4

Quality Control Inspector

python: 4

PDF Available

Summary

View CV

#2

H41k4l 455y4uq1

Score: 4

4.5.4. Pengujian 4

- Menggunakan keyword “servico” untuk menguji fuzzy matching

ATS CV Matcher

by stimastimastima

Welcome to Matchify - ATS CV Matcher!

Enter Keywords

servico

Select Algorithm

AC (Aho-Corasick)

1

50

Top CVs

10

Search CVs

Algorithm: AHO_CORASICK | Time: 537.6ms | Scanned: 479 CVs | Exact matches: 0 | Fuzzy: 1 keywords | Fuzzy Time: 3561.1ms

#1

Eva Kusmawati

Score: 2

Banking

servico (fuzzy): 5

PDF Available

Summary

View CV

4.6. Analisis Hasil Pengujian

Berdasarkan hasil pengujian terhadap berbagai algoritma pencocokan string yang digunakan dalam aplikasi, dapat disimpulkan bahwa masing-masing algoritma memiliki keunggulan dan efisiensi tersendiri, tergantung pada konteks pencarian yang dilakukan. Berikut adalah analisis performa dan kecocokan algoritma berdasarkan kasus dan kompleksitas waktu:

- Knuth-Morris-Pratt (KMP)

Algoritma ini cocok untuk panjang keyword yang pendek dan jumlah keyword yang tidak terlalu banyak. Kelebihan dari algoritma ini adalah memiliki kompleksitas waktu $O(n+m)$, dengan n adalah panjang teks dan m adalah panjang pola, karena menggunakan LPS (Longest Prefix Suffix) untuk menghindari pengulangan pencarian. Kekurangan dari algoritma ini adalah tidak terlalu efisien untuk pencarian dengan banyak keyword karena tidak didesain untuk itu.

- Boyer-Moore (BM)

Algoritma BM lebih cocok untuk kata kunci yang panjang, terutama jika jumlah karakter yang unik dan cukup besar. Algoritma ini memiliki kompleksitas waktu $O(n/m)$ pada *best case*, tetapi bisa $O(n \times m)$ pada *worst case*, meskipun secara praktis sering lebih cepat dari KMP untuk pola panjang. Kelebihan dari algoritma ini adalah dapat melakukan lompatan besar dalam teks jika terjadi ketidakcocokkan karakter yang membuatnya efisien dalam kasus umum. Kelemahannya adalah kurang cocok untuk keyword pendek atau jumlah pattern yang banyak.

- Aho-Corasick

Algoritma ini cocok untuk pencarian banyak keyword sekaligus dalam satu proses pencarian. Algoritma ini memiliki kompleksitas waktu $O(n + k + z)$ dengan n adalah panjang teks, k adalah total panjang semua keyword, dan z adalah jumlah total kecocokkan. Kelebihan dari algoritma ini adalah membangun automata (*trie + failure function*) sehingga semua kata kunci bisa dicari hanya dengan satu pemindaian teks. Kelemahan dari algoritma ini adalah memakan memori sedikit lebih besar karena membangun trie dan failure links.

- Levenshtein Distance (Fuzzy Matching)

Algoritma ini cocok untuk kasus pencarian kata kunci yang kemungkinan adanya *typo*. Algoritma ini memiliki kompleksitas waktu $O(n \times m)$ untuk membandingkan dua kata,

sehingga dapat memerlukan waktu yang cukup besar apabila teks yang dianalisis panjang atau jumlah keyword yang digunakan banyak.

Secara keseluruhan, pemilihan algoritma pencocokan sangat bergantung pada karakteristik pencarian yang dilakukan. Untuk pencarian dengan sedikit keyword dan tidak panjang, algoritma KMP memberikan hasil efisien. Sementara itu, Boyer-Moore lebih unggul saat menangani pola yang panjang. Jika jumlah kata kuncinya banyak, Aho-Corasick menjadi pilihan terbaik karena kemampuannya melakukan pencarian serentak dalam satu pemindaian. Levenshtein Distance memberikan fleksibilitas tambahan dalam menghadapi variasi penulisan atau *typo*, meskipun dengan lama waktu yang lebih tinggi. Dengan mengintegrasikan keempat algoritma ini, sistem dapat memberikan hasil pencarian yang relevan dan adaptif terhadap berbagai kondisi input pengguna.

BAB 5

KESIMPULAN, SARAN, DAN REFLEKSI

5.1. Kesimpulan

Dalam Tugas Besar 3 ini, kelompok *stimastimastima* berhasil membangun sistem ATS (*Applicant Tracking System*) berbasis CV digital bernama Matchify dengan bahasa Python, GUI Flet, dan database MySQL. Pencarian CV berdasarkan kata kunci masukan pengguna dilakukan dengan algoritma *string matching* sesuai keinginan pengguna, yaitu Knuth-Morris-Pratt (KMP), Boyer-Moore (BM), dan Aho-Corasick (AC), serta didukung algoritma Levenshtein Distance untuk *fuzzy match* jika kata kunci tidak ditemukan secara persis dalam seluruh file CV. Selain itu, ekstraksi file CV yang berbentuk PDF ke data yang dapat digunakan untuk pencocokan memanfaatkan *regex*. Setiap hasil pencarian dapat menampilkan detail lebih lanjut mengenai CV terkait dengan jumlah hasil pencarian yang ditampilkan berdasarkan masukan pengguna.

5.2. Saran

Saran dari kelompok terkait tugas besar ke depannya adalah perlunya *deadline* dipertimbangkan dengan lebih baik sehingga tidak mengganggu masa ujian. Akibatnya, mahasiswa dapat fokus memaksimalkan ujian. Selain itu, apabila terdapat pertanyaan di QnA yang dipandang penting, sebaiknya dimasukkan ke spesifikasi sebagai bentuk revisi.

5.3. Refleksi

Melalui tugas besar ini, kelompok mendapatkan pengetahuan yang lebih dalam mengenai *string matching*, sebab kami harus mengimplementasikannya. Secara khusus, kami mempelajari banyak sintaks *regular expression (regex)* baru serta menyadari betapa luasnya penggunaan *regex*. Kelompok juga diberikan kesempatan untuk mengeksplorasi lebih dalam mengenai Flet, sebuah GUI Python multifungsi yang ke depannya bermanfaat. Dari segi *soft skill*, tugas besar ini melatih kemampuan komunikasi dalam kelompok demi mewujudkan hasil yang maksimal. Kemampuan manajemen waktu kami juga dilatih karena tugas besar ini memiliki rentang pengerjaan yang beririsan dengan banyak tugas besar lain serta UAS.

LAMPIRAN

No	Poin	Ya	Tidak
1	Aplikasi dapat dijalankan.	✓	
2	Aplikasi menggunakan basis data berbasis SQL dan berjalan dengan lancar.	✓	
3	Aplikasi dapat mengekstrak informasi penting menggunakan Regular Expression (Regex).	✓	
4	Algoritma <i>Knuth-Morris-Pratt (KMP)</i> dan <i>Boyer-Moore (BM)</i> dapat menemukan kata kunci dengan benar.	✓	
5	Algoritma Levenshtein Distance dapat mengukur kemiripan kata kunci dengan benar.	✓	
6	Aplikasi dapat menampilkan <i>summary CV applicant</i> .	✓	
7	Aplikasi dapat menampilkan <i>CV applicant</i> secara keseluruhan.	✓	
8	Membuat laporan sesuai dengan spesifikasi.	✓	
9	Membuat bonus enkripsi data profil <i>applicant</i> .		✓
10	Membuat bonus algoritma Aho-Corasick.	✓	
11	Membuat bonus video dan diunggah pada Youtube.	✓	

Tautan Repository GitHub

https://github.com/naomirisaka/Tubes3_stimastimastima

Tautan Video

https://youtu.be/IVr7ymq_iuU

DAFTAR PUSTAKA

- Munir, R. 2025. “Pencocokan String (String Matching) dengan Algoritma Brute Force, KMP, Boyer-Moore”. [Online]. Tersedia: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/23-Pencocokan-string-\(2025\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/23-Pencocokan-string-(2025).pdf). [13 Juni 2025].
- Munir, R. 2025. “Pencocokan String dengan Regular Expression (Regex)”. [Online]. Tersedia: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/24-String-Matching-dengan-Regex-\(2025\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2024-2025/24-String-Matching-dengan-Regex-(2025).pdf). [13 Juni 2025].